MATERIAL DE ESTUDIO PARA DEFENSA ORAL

Guía Técnica de Defensa y Demostración en Vivo

Respuestas Clave, Paso a Paso de la Demo y Explicación del Código Modificado

Objetivo de esta Guía de Estudio

DEFENSA ORAL

- **Propósito:** Preparar a Jorge Mendoza, Yeremi Navarro y Renato Escarate para rendir con excelencia la defensa oral presencial de 15 minutos (5 min demo + 10 min preguntas cruzadas).
- **Enfoque:** Explicar con total claridad qué se modificó en el código, cómo funciona cada patrón y cómo argumentar frente al docente para obtener la nota máxima (5/5 pts en los 5 criterios).

1 Resumen de los Cambios Implementados en el Código

Para responder a los requisitos obligatorios de la pauta, se realizaron 5 intervenciones técnicas estratégicas en el código:

Mapa de Archivos Modificados e Intervenciones

CAMBIOS TÉCNICOS

- **1. punto_casino/models/product.py:** Se implementaron los métodos to_dict() y from_dict(). to_dict() transforma el objeto Producto en un diccionario estándar de Python listo para JSON. from_dict() permite reconstruir el objeto tipado.
- **2. punto_casino/models/order.py:** Se dotó a Order y OrderItem de to_dict() y from_dict(). to_dict() maneja fechas complejas convirtiendo created_at en string estándar con .isoformat() y serializa la lista de ítems de forma anidada.
- **3. punto_casino/repositories/product_repository.py:** Se evolucionó a JsonProductRepository. Ahora lee y escribe de forma permanente en data/productos.json usando with open(..., encoding='utf-8') y json.dump(indent=4)/json.load().
- **4. punto_casino/repositories/order_repository.py:** Se implementó JsonOrderRepository. Guarda y sincroniza todas las comandas en data/pedidos.json. Mide el tiempo de I/O en disco con time.perf_counter().
- **5. punto_casino/services/order_service.py:** En get_accounting_report() se integró NumPy (np.mean, np.median, np.std) para calcular la media, mediana y dispersión del ticket de compra, junto con telemetría de milisegundos.
- **6. punto_casino/views/screens/admin_screen.py:** En la pantalla de Administrador se agregaron 3 badges visuales que muestran la Mediana (NumPy), Desv. Estándar (NumPy) y el tiempo de cómputo en ms.
- **7. tests/test_domain.py:** Se añadieron 4 pruebas unitarias automatizadas (13 en total, 100% aprobadas) que validan to_dict(), from_dict(), persistencia JSON y cálculos NumPy.

2 Guión de la Demostración en Vivo (5 Minutos)

La pauta de evaluación exige explícitamente: 'Demostración en Vivo (5 min): El grupo ejecuta la app, demuestra la carga inicial desde el JSON, registra un nuevo elemento (evidenciando la fecha automática) y comprueba la actualización del archivo en disco.' Sigan este guión exacto frente al profesor:

Paso 1: Evidenciar el Fichero JSON en VS Code antes de Abrir la App (30 seg)

PASO 1

- **Acción:** Abran VS Code y muestren la carpeta 'data/'. Abran el archivo 'data/pedidos.json'.
- **Qué decirle al profesor:** 'Profesor, aquí en data/pedidos.json tenemos almacenadas las comandas en formato estructurado JSON. Fíjese en el último pedido registrado (ej: comanda #107) y que cada registro incluye su timestamp ISO.'

Paso 2: Ejecutar la App y Demostrar la Carga Inicial (1 min)

PASO 2

- **Acción:** Ejecuten en la terminal: './kivy_env/bin/python3 main.py' (o desde VS Code con F5).
- **Qué decirle al profesor:** 'Al iniciar, la app ejecuta JsonProductRepository y JsonOrderRepository, que leen con json.load() y with open(..., encoding="utf-8") los datos en disco, reconstruyendo los objetos con from_dict().'
- **En pantalla:** Hagan login rápido como 'Estudiante' o 'Cajero' y muestren que las comandas y platos ya están cargados.

Paso 3: Registrar un Nuevo Pedido y Evidenciar Fecha Automática (1.5 min)

PASO 3

- **Acción:** Entren como Estudiante -> Catálogo -> Agreguen un plato (ej. 'Pastel de Choclo') al carrito -> Toquen 'Pagar' -> Seleccionen 'Beca BAES JUNAEB'.
- **Qué mostrar en la UI:** La comanda se genera de inmediato con un nuevo número correlativo (#108), estado CONFIRMED y marca de tiempo automática generada con datetime.now() visible en la tarjeta (ej. 14:48 hrs).
- **Qué decirle al profesor:** 'El timestamp no es estático ni hardcodeado: lo asigna automáticamente la entidad Order al instanciarse en la capa de modelos.'

Paso 4: Comprobar la Actualización Inmediata del Archivo en Disco (1 min)

PASO 4

- **Acción:** Vuelvan a la ventana de VS Code con 'data/pedidos.json' abierto y bajen hasta el final del archivo.
- **Qué mostrar:** El nuevo pedido #108 aparece escrito en el JSON con su id_pedido, items, created_at y total.
- **Qué decirle al profesor:** 'En el momento en que se completó la compra, el servicio invocó a order_repo.add(), el cual ejecutó _save_to_json() con with open() y json.dump(indent=4), persistiendo la información de forma segura.'

Paso 5: Mostrar la Integración de NumPy y time.perf_counter() en Admin (1 min)

PASO 5

- **Acción:** En la app móvil, toquen 'Salir' e inicien sesión rápida como 'Administrador' (admin@uct.cl).
- **Qué mostrar:** Muestren la tarjeta superior 'Resumen Contable del Casino'. Señalen las pastillas morada y naranja: 'Mediana (NumPy)' y 'Desv. Est. (NumPy)', además del tiempo en ms de perf_counter().
- **Qué decirle al profesor:** 'Aquí integramos NumPy como librería científica del stack para calcular analítica de compras en tiempo real: convertimos los totales a un arreglo vectorial NumPy y calculamos media, mediana y desviación estándar en menos de 1 ms medido con perf_counter().'

3 Ronda de Preguntas Técnicas del Docente (10 Minutos)

El docente realizará preguntas cruzadas para comprobar el dominio del código fuente. Aquí tienen las respuestas exactas:

Pregunta 1: ¿Por qué usaron 'with open()' y qué pasa si no se usa?

PREGUNTA 1

- **Respuesta recomendada:** 'Usamos with open() porque implementa el patrón Context Manager en Python (__enter__ y __exit__). Garantiza que el descriptor de archivo en el sistema operativo se cierre inmediatamente al finalizar el bloque, incluso si ocurre una excepción como falta de memoria o error de parseo. Si usáramos open() simple sin with y la app crashea a mitad de escritura, el archivo queda bloqueado, los buffers no se vacían (flush) y el archivo JSON se corrompe.'

Pregunta 2: ¿Por qué es necesario el método to_dict() en vez de guardar el objeto directo?

PREGUNTA 2

- **Respuesta recomendada:** 'El serializador nativo json.dump() de Python solo sabe convertir tipos primitivos (dict, list, str, int, float, bool). Nuestras clases de dominio Product y Order contienen objetos complejos como enumeraciones (OrderStatus, PaymentMethod) e instancias datetime. El método to_dict() se encarga de transformar esos tipos complejos en representaciones primitivas estándar (como strings ISO 8601), desacoplando además el modelo de negocio del formato de almacenamiento.'

Pregunta 3: ¿Por qué la carpeta services/ debe estar 100% libre de Kivy?

PREGUNTA 3

- **Respuesta recomendada:** 'Es la base del principio de Separación de Responsabilidades (SoC) y Clean Architecture. Si la lógica de negocio importara widgets de Kivy o KivyMD, no podríamos ejecutar pruebas unitarias automáticas sin levantar ventanas gráficas, ni podríamos reutilizar la lógica de negocio en una futura versión web o de consola. Al mantener services/ en Python puro, ejecutamos tests/test_domain.py en segundos en cualquier entorno.'

Pregunta 4: ¿Qué función real cumple NumPy en su proyecto?

PREGUNTA 4

- **Respuesta recomendada:** 'NumPy no está puesto como un adorno ni en un print suelto: procesa el vector contable de ventas en la capa de servicios (get_accounting_report()). Cuando el casino tiene decenas o cientos de comandas, NumPy realiza operaciones vectorizadas en memoria en lenguaje C, calculando la mediana de consumo (que es más representativa que la media porque no se distorsiona con compras gigantes atípicas) y la desviación estándar para analizar la dispersión del gasto de los estudiantes.'

Pregunta 5: ¿Cómo garantizan que las fechas no se desconfiguren al guardar y leer?

PREGUNTA 5

- **Respuesta recomendada:** 'Al persistir, usamos created_at.isoformat(), que genera un estándar internacional sin ambigüedad horaria (YYYY-MM-DDTHH:MM:SS). Al cargar desde el JSON con from_dict(), usamos datetime.fromisoformat(), lo que reconstruye un objeto datetime formal de Python. Así podemos seguir realizando operaciones temporales como ordenar pedidos cronológicamente o calcular tiempos de espera.'

4 Glosario Rápido de Conceptos Clave para la Defensa

Términos técnicos que deben usar con naturalidad durante su exposición oral:

Vocabulario Técnico Esencial

GLOSARIO

- **SoC (Separation of Concerns):** Separación de responsabilidades: dividir el programa en módulos donde cada uno tiene una única razón para cambiar (UI en views, lógica en services, datos en models/repos).
- **Serialización:** Proceso de convertir un objeto en memoria (instancia de clase) a un formato de texto transmisible o almacenable (JSON) mediante to_dict(). Deserialización es el proceso inverso con from_dict().
- **Context Manager (with open):** Protocolo de Python que asegura la adquisición y liberación determinista de recursos del sistema operativo.
- **UTF-8 (encoding):** Codificación de caracteres estándar que asegura que tildes, ñ y caracteres en español se guarden y lean sin romper el archivo ni mostrar caracteres extraños (mojibake).
- **Vectorización (NumPy):** Capacidad de aplicar operaciones matemáticas sobre arreglos continuos de memoria sin usar bucles for manuales en Python, alcanzando velocidades nativas de C.
- **time.perf_counter():** Reloj monotónico de alta resolución en Python diseñado específicamente para medir intervalos de tiempo de ejecución sin verse afectado por ajustes del reloj del sistema.

5 Estrategia para Obtener 5/5 en Cada Criterio de la Rúbrica

Criterio de Evaluación	Qué exige la Rúbrica para Excelente (5 pts)	Cómo lo demostramos en la app
Arquitectura Modular (S…	Estructura impecable (views, services, models) con s…	Mostrar carpetas; services/ tiene 0 imports de Kivy; main.py so…
Persistencia JSON y POO	Clases POO robustas con to_dict() y persistencia lim…	Mostrar Product y Order con to_dict(), y la creación en vivo en …
Manejo de Ficheros y F…	Uso correcto de with open(encoding='utf-8') y timesta…	Mostrar el código de _save_to_json() con with open y fecha IS…
Librería Adicional	Librería adicional integrada aportando valor real al flu…	Mostrar en pantalla Admin la Mediana y Desv. Est. calculadas …
Informe Técnico y Defe…	Informe técnico impecable y dominio total del código …	Entregar el PDF de 5 páginas y responder las preguntas con e…

Consejo Final para el Equipo

¡ÉXITO EN LA DEFENSA!

- **División del Equipo:** Jorge puede abrir la demo e introducir la arquitectura (SoC); Yeremi puede explicar el modelado POO, to_dict() y el guardado en JSON; Renato puede explicar la analítica con NumPy y los timestamps.
- **Seguridad y Confianza:** El proyecto cuenta con 13 pruebas unitarias aprobadas, código limpio y dos informes formales. ¡Tienen todo listo para obtener la nota máxima!